

Top 50 SQL Interview Questions with Answers

1. Find the Second Highest Salary

Scenario: In HR departments, it's common to identify the second-highest earner for promotion decisions or salary benchmarking. This query helps find the person earning the second-highest salary without duplicates or considering ties.

Answer

```
SELECT MAX(salary)
FROM Employee
WHERE salary <
(
    SELECT MAX(salary)
    FROM Employee
);
```

2. Find Nth Highest Salary

Scenario: Management needs flexible access to salary tiers (5th, 10th, etc.) for compensation analysis. This query allows finding any salary rank dynamically by replacing N with a number.

Answer

```
SELECT salary
FROM
(
    SELECT salary,
           DENSE_RANK() OVER(ORDER BY salary DESC) rnk
    FROM Employee
) t
WHERE rnk = N;
```

3. Highest Salary in Each Department

Scenario: Finance teams need to understand departmental salary structures for budget allocation and compliance. This query shows the maximum earner in each department for comparative analysis.

Answer

```
SELECT department,
       MAX(salary)
FROM Employee
GROUP BY department;
```

4. Second Highest Salary in Each Department

Scenario: Department heads want to identify succession candidates or understand salary gaps. This identifies the second-highest earner in each department separately.

Answer

```
SELECT *
FROM
(
    SELECT *,
           DENSE_RANK() OVER
           (
               PARTITION BY department
               ORDER BY salary DESC
           ) rnk
    FROM Employee
) t
WHERE rnk=2;
```

5. Top 3 Highest Salaries in Each Department

Scenario: For talent management and retention analysis, companies track top earners in each department. This query identifies the top 3 performers in every department to assess key personnel distribution.

Answer

```
SELECT *
FROM
(
  SELECT *,
    DENSE_RANK() OVER
    (
      PARTITION BY department
      ORDER BY salary DESC
    ) rnk
  FROM Employee
)t
WHERE rnk<=3;
```

6. Employees Earning More Than Department Average

Scenario: HR uses this to identify high performers earning above their department's average for performance evaluations and reward justifications.

Answer

```
SELECT *
FROM Employee e
WHERE salary >
(
  SELECT AVG(salary)
  FROM Employee
  WHERE department=e.department
);
```

7. Employees Earning Less Than Department Average

Scenario: During compensation review cycles, HR identifies employees below departmental average to assess whether they need salary adjustments or additional training.

Answer

```
SELECT *  
FROM Employee e  
WHERE salary <  
(  
SELECT AVG(salary)  
FROM Employee  
WHERE department=e.department  
);
```

8. Employee Having Highest Salary

Scenario: Executive management needs to know the top earner in the organization for various reporting and governance purposes.

Answer

```
SELECT *  
FROM Employee  
ORDER BY salary DESC  
LIMIT 1;
```

9. Employee Having Lowest Salary

Scenario: Compliance teams verify minimum wage standards or identify entry-level employees for mentorship programs.

Answer

```
SELECT *  
FROM Employee  
ORDER BY salary  
LIMIT 1;
```

10. Find Duplicate Records

Scenario: Data quality checks are essential before merging customer records or processing bulk operations. This identifies duplicate email entries that might indicate data entry errors or account abuse.

Answer

```
SELECT email,  
COUNT(*)  
FROM Employee  
GROUP BY email  
HAVING COUNT(*)>1;
```

11. Delete Duplicate Records

Scenario: After identifying duplicates (Question 10), organizations need to clean data by removing redundant entries while keeping the earliest record. This maintains data integrity for critical systems.

Answer

```
DELETE FROM Employee  
WHERE id NOT IN  
(  
SELECT MIN(id)  
FROM Employee  
GROUP BY email  
);
```

12. Find Employees Without Manager

Scenario: Organizational structure validation requires identifying top-level executives or orphaned records where manager assignment is missing. This ensures reporting hierarchy completeness.

Answer

```
SELECT *  
FROM Employee  
WHERE managerid IS NULL;
```

13. Employees Earning More Than Their Manager

Scenario: Compensation audits flag organizational anomalies where subordinates earn more than supervisors, indicating potential role misclassification or salary compression issues.

Answer

```
SELECT e.*  
FROM Employee e  
JOIN Employee m  
ON e.managerid=m.employeeid  
WHERE e.salary>m.salary;
```

14. Employees Joined in Last 30 Days

Scenario: HR onboarding teams track recent hires for training program assignment, orientation scheduling, and probation monitoring.

Answer

```
SELECT *  
FROM Employee  
WHERE joiningdate>=CURRENT_DATE-INTERVAL '30 day';
```

15. Count Employees in Each Department

Scenario: Resource planning and departmental budget allocation depend on headcount analysis. This provides the basis for staffing ratios and resource distribution decisions.

Answer

```
SELECT department,  
COUNT(*)  
FROM Employee  
GROUP BY department;
```

16. Find Departments Having More Than 10 Employees

Scenario: Large departments may require restructuring or sub-team creation. Management uses this to identify departments that exceed staffing thresholds for organizational optimization.

Answer

```
SELECT department,  
COUNT(*)  
FROM Employee  
GROUP BY department  
HAVING COUNT(*)>10;
```

17. Find Duplicate PAN Numbers

Scenario: India's tax identification (PAN) compliance requires unique entries per person. Financial and legal teams use this to detect multiple accounts for the same individual, preventing tax fraud.

Answer

```
SELECT pannumber,  
COUNT(*)  
FROM Customer  
GROUP BY pannumber  
HAVING COUNT(*)>1;
```

18. Find Missing IDs

Scenario: When IDs should be sequential (1, 2, 3...) but aren't, gaps indicate deleted records or data entry issues. Database admins use this for auditing and integrity checks.

Answer

```
SELECT id+1  
FROM Employee  
WHERE id+1 NOT IN  
(  
SELECT id  
FROM Employee  
);
```

19. ROW_NUMBER()

Scenario: Analytics teams need unique row identifiers for pagination, ranking without considering ties, or selecting top N records per group. ROW_NUMBER assigns 1, 2, 3... even for identical values.

Assigns unique numbers.

Answer

```
SELECT *,  
ROW_NUMBER() OVER(ORDER BY salary DESC)  
FROM Employee;
```

20. RANK()

Scenario: Sports leaderboards, competition rankings, or salary tiers often require handling ties (rank 1, 2, 2, 4...). RANK() assigns the same rank to tied values and skips numbers.

Answer

```
SELECT *,  
RANK() OVER(ORDER BY salary DESC)  
FROM Employee;
```

21. DENSE_RANK()

Scenario: Unlike RANK(), DENSE_RANK() doesn't skip numbers when ties occur (1, 2, 2, 3...). Used when you need continuous ranking sequences without gaps, such as in academic grading systems.

Answer

```
SELECT *,  
DENSE_RANK() OVER(ORDER BY salary DESC)  
FROM Employee;
```

22. LAG()

Scenario: Time-series analysis requires comparing current values with previous records. Finance uses this to calculate month-over-month salary changes or identify anomalies in sequential data.

Answer

```
SELECT employeeid,  
salary,  
LAG(salary) OVER(ORDER BY salary)  
FROM Employee;
```

23. LEAD()

Scenario: Forecasting and predictive analysis use LEAD() to access future values. Sales teams use this to compare current quarter performance against next quarter targets.

Answer

```
SELECT employeeid,  
salary,  
LEAD(salary) OVER(ORDER BY salary)  
FROM Employee;
```

24. FIRST_VALUE()

Scenario: Baseline comparisons require fetching the first value in a window. Analytics use this to compare each employee's current salary to their starting salary for growth analysis.

Answer

```
SELECT *,  
FIRST_VALUE(salary)  
OVER(ORDER BY salary DESC)  
FROM Employee;
```

25. LAST_VALUE()

Scenario: Cumulative analysis requires the final value in a window frame. Accounting uses this to calculate total expenses or revenue at the end of a period.

Answer

```
SELECT *,  
LAST_VALUE(salary)  
OVER(  
ORDER BY salary  
ROWS BETWEEN UNBOUNDED PRECEDING  
AND UNBOUNDED FOLLOWING  
)  
FROM Employee;
```

26. Running Total

Scenario: Financial reports and dashboards require cumulative sums over time. Banks use this to show account balance progression or invoice payment history.

Answer

```
SELECT employeeid,  
salary,  
SUM(salary)  
OVER(ORDER BY employeeid)  
FROM Employee;
```

27. Moving Average

Scenario: Data smoothing for trend analysis requires calculating averages over rolling windows. Stock traders use 3-day, 7-day, or 30-day moving averages to identify trends and reduce noise.

Answer

```
SELECT salary,  
AVG(salary)  
OVER(  
ORDER BY employeeid  
ROWS BETWEEN 2 PRECEDING  
AND CURRENT ROW  
)  
FROM Employee;
```

28. Find Customers Without Orders

Scenario: E-commerce companies identify inactive or prospect customers who haven't made purchases. Marketing teams use this for targeted outreach campaigns or subscription retention programs.

Answer

```
SELECT *  
FROM Customer c  
LEFT JOIN Orders o  
ON c.customerid=o.customerid  
WHERE o.customerid IS NULL;
```

29. Find Orders Without Customer

Scenario: Data integrity checks reveal orphaned orders from deleted customer records. This indicates referential integrity issues that require database cleanup or transaction rollback.

Answer

```
SELECT *  
FROM Orders o  
LEFT JOIN Customer c  
ON o.customerid=c.customerid  
WHERE c.customerid IS NULL;
```

30. Inner Join Example

Scenario: Only matching records between customers and orders are needed for revenue analysis. An INNER JOIN shows only customers who have placed orders, excluding inactive accounts.

Answer

```
SELECT *  
FROM Customer  
INNER JOIN Orders  
USING(customerid);
```

31. Left Join

Scenario: Complete customer lists with orders (even missing ones) are needed for customer lifetime value analysis. A LEFT JOIN retains all customers while adding order data when available.

Answer

```
SELECT *  
FROM Customer  
LEFT JOIN Orders  
USING(customerid);
```

32. Right Join

Scenario: Supplier management sometimes uses RIGHT JOIN to prioritize one table. E.g., ALL suppliers visible with their products, or ALL orders visible with matching customers.

Answer

```
SELECT *  
FROM Customer  
RIGHT JOIN Orders  
USING(customerid);
```

33. Full Join

Scenario: Reconciliation processes require matching records from two sources with complete visibility. A FULL JOIN shows matched, unmatched from left, AND unmatched from right—essential for data migration validation.

Answer

```
SELECT *  
FROM Customer  
FULL JOIN Orders  
USING(customerid);
```

34. Cross Join

Scenario: Generating combinations like all possible employee-department pairings for assignment matrices, or every product with every store for location analysis. CROSS JOIN produces the Cartesian product.

Answer

```
SELECT *  
FROM Employee  
CROSS JOIN Department;
```

35. Self Join

Scenario: Organizational hierarchies require comparing rows within the same table. A self join shows each employee with their manager's details for reporting chain analysis.

Answer

```
SELECT e.employeeid,  
m.employeeid manager  
FROM Employee e  
LEFT JOIN Employee m  
ON e.managerid=m.employeeid;
```

36. EXISTS Example

Scenario: Performance-optimized queries use EXISTS instead of IN for large datasets. E.g., finding customers with at least one order is faster using EXISTS as it stops searching after finding the first match.

Answer

```
SELECT *  
FROM Customer c  
WHERE EXISTS  
(  
SELECT 1  
FROM Orders o  
WHERE o.customerid=c.customerid  
);
```

37. NOT EXISTS Example

Scenario: Negative logic queries efficiently find records without related data. Using NOT EXISTS to find customers without orders is more performant than using LEFT JOIN with NULL checks in many databases.

Answer

```
SELECT *  
FROM Customer c  
WHERE NOT EXISTS  
(  
SELECT 1  
FROM Orders o  
WHERE o.customerid=c.customerid  
);
```

38. UNION vs UNION ALL

Scenario: Combining results from multiple queries (customer cities and supplier cities) requires UNION for unique results or UNION ALL for performance when duplicates are acceptable. UNION removes duplicates at a performance cost.

Answer

```
SELECT city FROM Customer
UNION
SELECT city FROM Supplier;
```

39. Common Table Expression (CTE)

Scenario: Complex queries with repetitive logic become readable using CTEs. E.g., creating reusable "employee rankings" subquery makes the main query cleaner and more maintainable.

Answer

```
WITH cte AS
(
  SELECT *,
  ROW_NUMBER() OVER(ORDER BY salary DESC) rn
  FROM Employee
)
SELECT *
FROM cte;
```

40. Recursive CTE

Scenario: Hierarchical queries like organizational charts, bill-of-materials, or comment threads use recursive CTEs. Starting with top-level items, recursion fetches child items repeatedly until no more children exist.

Answer

```
WITH RECURSIVE nums AS
(
  SELECT 1 n
  UNION ALL
  SELECT n+1
  FROM nums
  WHERE n<10
)
SELECT *
FROM nums;
```

41. Pivot Data

Scenario: Reports often require transforming rows into columns for readability. E.g., department budgets showing each month as a column instead of rows makes comparison easier for executive dashboards.

Use conditional aggregation.

Answer

```
SELECT
SUM(CASE WHEN gender='M' THEN 1 END) Male,
SUM(CASE WHEN gender='F' THEN 1 END) Female
FROM Employee;
```

42. Find Even Records

Scenario: Sampling or batch processing requires selecting alternating records (even IDs: 2, 4, 6...). This is useful for A/B testing, load distribution, or quality sampling without selecting every record.

Answer

```
SELECT *
FROM Employee
WHERE MOD(employeeid,2)=0;
```

43. Find Odd Records

Scenario: Complementary to finding even records, odd ID queries (1, 3, 5...) serve the same sampling purpose. Combined with even records, you can process data in two parallel batches.

Answer

```
SELECT *
FROM Employee
WHERE MOD(employeeid,2)=1;
```

44. Latest Record per Customer

Scenario: Customer service teams need the most recent order per customer for quick reference. E.g., showing last purchase date, order status, or delivery address for support interactions.

Answer

```

SELECT *
FROM
(
  SELECT *,
  ROW_NUMBER() OVER
  (
    PARTITION BY customerid
    ORDER BY createddate DESC
  ) rn
FROM Orders
)t
WHERE rn=1;

```

45. Find Consecutive Records

Scenario: Identifying sequences of consecutive IDs helps detect data gaps or logical groups. E.g., finding employee groups hired consecutively or transaction sequences for fraud detection.

Answer

```

SELECT *
FROM
(
  SELECT *,
  id-ROW_NUMBER() OVER(ORDER BY id) grp
FROM Employee
)t;

```

46. Difference Between RANK(), DENSE_RANK(), ROW_NUMBER()

Scenario: Interview questions distinguish these three functions: ROW_NUMBER assigns 1,2,3 (no ties), RANK assigns 1,2,2,4 (skips), DENSE_RANK assigns 1,2,2,3 (no skip). Choosing the right one depends on whether tied values matter and whether you want gaps.

Function	Duplicate Rank	Skip Rank	ROW_NUMBER	RANK	DENSE_RANK
1,2,2,4	Yes	No	No	Yes	Yes
1,2,2,3	No	Yes	No	No	Yes

47. DELETE vs TRUNCATE vs DROP

Scenario: Database maintenance requires understanding these operations: DELETE removes selected rows (can use WHERE, slow, rollbackable); TRUNCATE removes all rows fast (resets identity, minimally logged);

DROP removes the table structure entirely (DDL, irreversible). Each has specific use cases.

DELETE	TRUNCATE	DROP
Removes selected rows	Removes all rows	Removes table
Can use WHERE	No WHERE	Deletes structure
Rollback possible (transaction dependent)	Often minimally logged	Removes object

48. Clustered vs Non-Clustered Index

Scenario: Index selection is critical for query performance. A clustered index (physical order) sorts the table data and exists once per table. Non-clustered indexes (separate pointers) can be created multiple times for different columns to optimize various queries.

Clustered

- Data stored in index order
- One per table

Non-Clustered

- Separate index structure
- Multiple allowed

49. Primary Key vs Unique Key

Scenario: Constraint design determines data integrity: Primary Keys uniquely identify each row and cannot be NULL (one per table). Unique Keys enforce uniqueness but allow multiple NULLs. For example, Email (unique, nullable) vs EmployeeID (primary, not nullable).

Primary	Unique	--	No NULL	NULL allowed (DB-dependent)	One per table	Multiple	Uniquely identifies row	Enforces uniqueness
---------	--------	----	---------	-----------------------------	---------------	----------	-------------------------	---------------------

50. Explain SQL Execution Order

Scenario: Understanding logical execution order is crucial for writing efficient queries and debugging unexpected results. For example, WHERE filters before GROUP BY, and HAVING filters after GROUP BY. This knowledge helps optimize WHERE conditions vs HAVING conditions.

Logical execution order:

1. FROM
2. JOIN
3. WHERE
4. GROUP BY
5. HAVING
6. SELECT
7. DISTINCT
8. ORDER BY
9. LIMIT / OFFSET

Bonus Advanced Questions

- What are Window Functions?
- Explain ACID Properties.
- What is Normalization?
- Explain Denormalization.
- What is Indexing?
- Explain Composite Index.
- What is Covering Index?
- What is Query Optimization?
- What is Execution Plan?
- What are Materialized Views?
- Difference between CHAR and VARCHAR.
- What are Correlated Subqueries?
- Explain COALESCE(), NULLIF(), CASE.
- What are Recursive CTEs?
- What are Transactions?
- Explain Locks (Shared, Exclusive).
- Deadlock vs Blocking.
- Partitioning vs Sharding.
- OLTP vs OLAP.
- How to optimize a slow SQL query?

These 50 questions cover the majority of SQL interview topics for **2–8 years of experience**, especially for PostgreSQL, SQL Server, MySQL, Oracle, and cloud data engineering roles. Top